Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

Department of CSE

B.Tech. CSE Trimester -5

Roll Number:2520030609
Name:V.Siri Chandana

Generative AI and AI Platforms

25CS2207E

WEEK 2 – GENERATIVE AI

Problem: Build a provider-agnostic LLM client: one interface over a hosted API and a local Ollama model

Solution:

1. Aim

To develop a provider-agnostic Large Language Model (LLM) client using Python that supports both a hosted Gemini API and a local Ollama model through a common interface.

2. Problem Statement

To build a common LLM client that can communicate with different AI providers, such as Gemini and Ollama, using the same interface.

3. Software Requirements

1. Python
2. PyCharm / Visual Studio Code
3. Ollama
4. Google Gemini API
5. Python libraries:
    - python-dotenv
    - pytest
    - google-genai
    - requests

4. Theory

Generative AI is a type of Artificial Intelligence that generates new content such as text, images and code.

A Large Language Model (LLM) understands user prompts and generates meaningful responses.

Ollama allows us to run AI models locally on our computer, whereas Gemini is a cloud-based AI service.

A provider-agnostic LLM client provides a common interface to communicate with different AI providers without changing the main application code.

5. Procedure

1. Downloaded and installed Ollama on the local system.
2. Opened Windows PowerShell and checked the Ollama version using ollama --version.
3. Downloaded the Llama 3.2 model using ollama pull llama3.2.
4. Created a project folder named week-2.
5. Created the following files:
    - .env
    - llm_client.py
    - main.py
    - test_client.py
    - requirements.txt
6. Installed the required Python libraries using pip install -r requirements.txt.
7. Created a common LLM provider interface using an abstract class.
8. Implemented the Gemini provider to communicate with the hosted Gemini API.
9. Implemented the Ollama provider to communicate with the locally running AI model.
10. Created the common LLM client to generate responses using the selected provider.
11. Executed main.py and selected Ollama as the provider.
12. Entered a question and received the AI-generated response.
13. Created a unit test using a fake provider to test the LLM client.

6.Code Snippets

llm_client.py

```
from abc import ABC, abstractmethod

from google import genai

import requests
```

```
        # Common interface
        class LLMProvider(ABC):

@abstractmethod
    def generate(self, prompt):
            pass

    # Gemini cloud provider
    class GeminiProvider(LLMProvider):

    def __init__(self, api_key):
            self.client = genai.Client(api_key=api_key)

    def generate(self, prompt):
            response = self.client.models.generate_content(
                    model="gemini-3.8-flash",
                    contents=prompt
            )
            return response.text

    # Ollama local provider
    class OllamaProvider(LLMProvider):

    def __init__(self, model="llama3.2"):
            self.model = model
            self.url = "http://localhost:11434/api/generate"

    def generate(self, prompt):
```

```
            response = requests.post(
                self.url,
            json={
                    "model": self.model,
                    "prompt": prompt,
                    "stream": False
                },
                timeout=120
            )

            response.raise_for_status()
            return response.json()["response"]

    # Common client
    class LLMClient:

    def __init__(self, provider):
            self.provider = provider

    def generate(self, prompt):
            return self.provider.generate(prompt)
```

main.py

```
    import os
    from dotenv import load_dotenv

    from llm_client import (
    GeminiProvider,
```

```
OllamaProvider,
LLMClient
)

load_dotenv(override=True)

provider_name = input(
"Choose provider (gemini/ollama): "
).strip().lower()

if provider_name == "gemini":

api_key = os.getenv("GEMINI_API_KEY")

if not api_key:
        raise ValueError("Gemini API key not found in .env")

provider = GeminiProvider(api_key)

elif provider_name == "ollama":

provider = OllamaProvider("llama3.2")

else:
raise ValueError("Invalid provider selected")

client = LLMClient(provider)
```

```
prompt = input("Enter your question: ")

answer = client.generate(prompt)

print("\nLLM Response:")
print(answer)
```

test_client.py

```
import unittest
from llm_client import LLMClient

# Fake provider for testing
class FakeProvider:

def generate(self, prompt):
        return "AI stands for Artificial Intelligence."

class TestLLMClient(unittest.TestCase):

def test_generate(self):

        provider = FakeProvider()
        client = LLMClient(provider)

        result = client.generate("Explain AI")

        self.assertEqual(
            result,
```

"AI stands for Artificial Intelligence."

)

if __name__ == "__main__":

unittest.main()

requirements.txt

python-dotenv

pytest

google-genai

requests

.env

GEMINI_API_KEY="Give your API Key here..."

pip install requirements.py

7. Observation

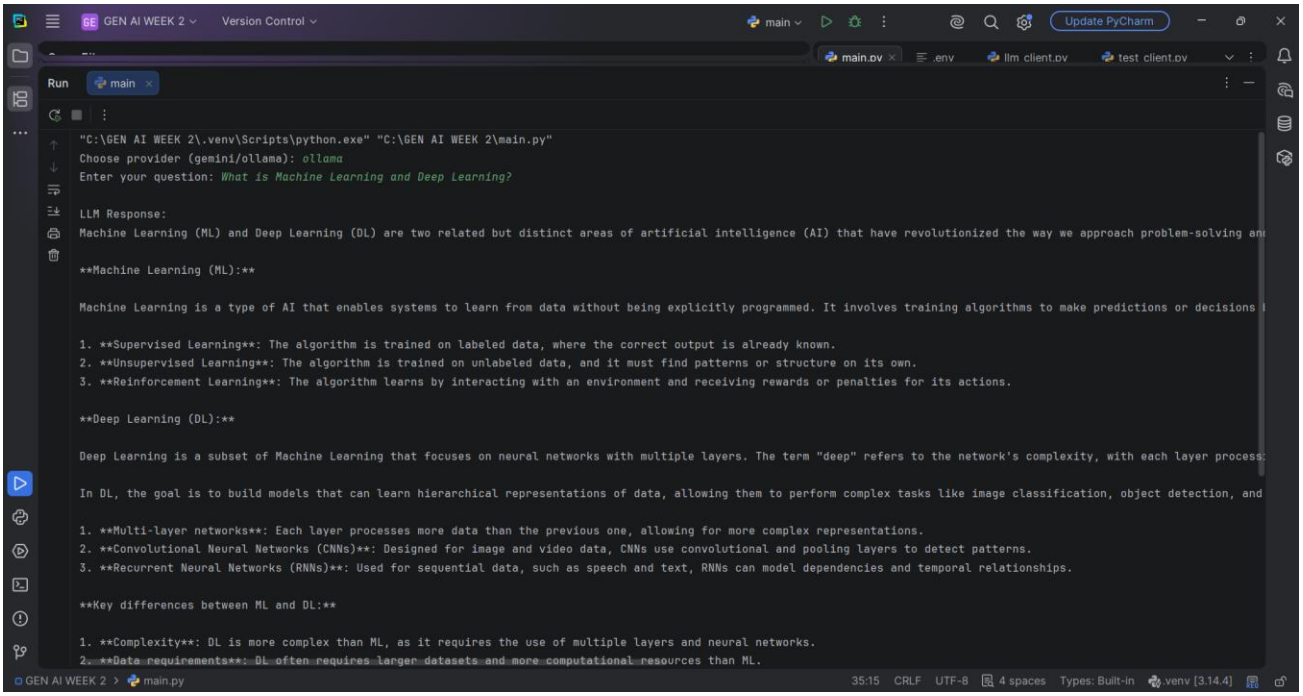
1. Ollama was successfully installed on the local system.
2. The Llama model was downloaded and used locally.
3. The application accepted questions from the user as prompts.
4. The selected LLM provider processed the prompt and generated a response.
5. The common interface supported communication with different AI providers.
6. Ollama generated responses without requiring a Gemini API key.
7. The fake provider test was designed to check whether the LLM client generates the expected response.

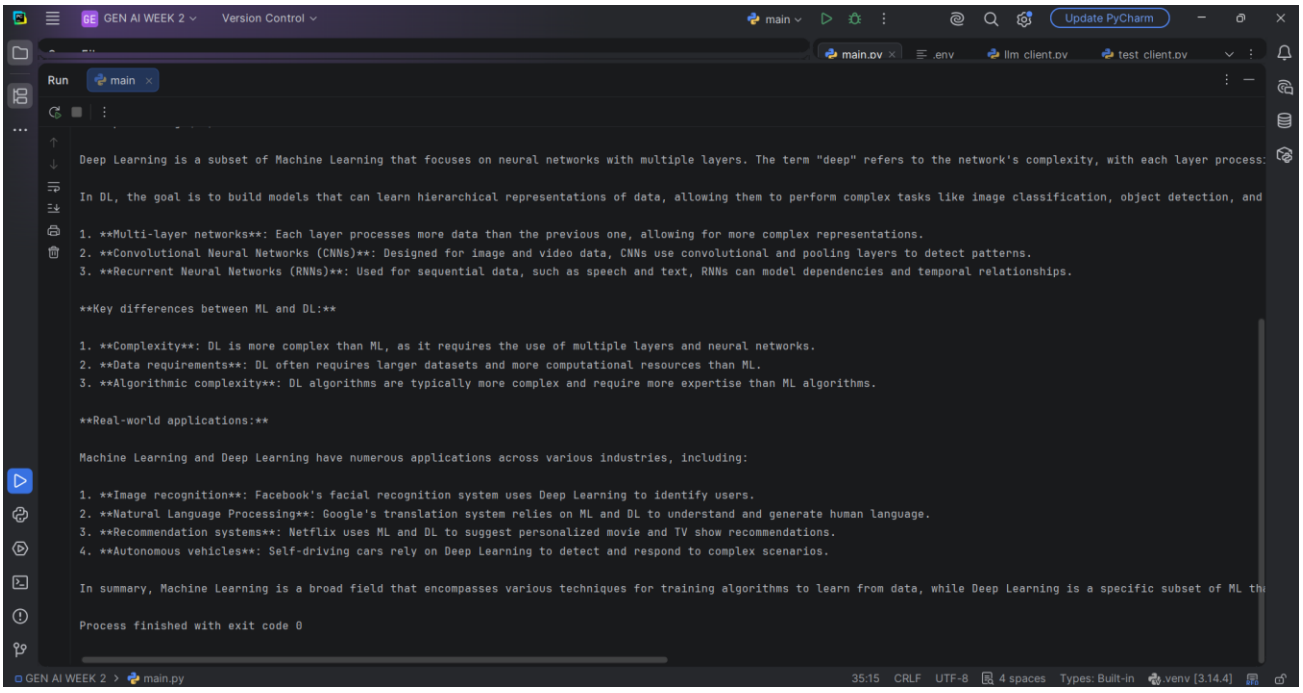
8. Result

Successfully developed a provider-agnostic LLM client using Python. The application supports both hosted Gemini and local Ollama providers through a common interface and generates AI responses based on user prompts.

9.Output

Using ollama





Using Gemini

